Shopizer E-Commerce Platform

Part 4. Continuous Integration.

Course: Software Testing

Student: Yijun Sun, Yu Chien Chiu

Date: February 23, 2026

Section 1: Continuous Integration (CI) and Its Purpose

Continuous Integration (CI) is a software development practice where developers frequently merge their code changes into a shared repository, usually multiple times a day. Each integration is verified by an automated build and automated tests to detect integration errors as quickly as possible.

The primary objectives of CI include:

- **Early Defect Detection**: By triggering automated tests after every push, integration conflicts or logic bugs can be identified immediately, preventing issues from accumulating into the later stages of development.
- **Ensuring Code Quality**: Executing test suites in a clean, human-intervention-free cloud environment (such as GitHub Actions runners) ensures that all code adheres to predefined structural and quality standards.
- **Accelerated Release Cycle**: Automating repetitive compilation and testing tasks significantly reduces the time cost of manual verification, allowing the team to iterate more confidently and rapidly.
- **Establishing a Reliable Baseline**: Ensuring the "shared branch" remains in a stable state—always buildable and passing tests—provides a trustworthy foundation for team collaboration.

Section 2: CI Assignment Error Log

Record date: 2026-02-18
Goal: capture each build/test error step-by-step for assignment evidence (process + issues + handling).

Logging Rules

- Add one entry for every key step (setup, build, test, push, CI trigger).
- Even when a step succeeds, record it as "Success (no error)" for traceability.
- For failures, always capture: command, exit code, error summary, initial analysis, and next action.

Error Timeline (Chronological)

Step 1: Run test via VS Code test tool

- **Action**: run `ProductPricePartitionTest.java`
- **Result**: no actual Java test execution stats (`passed=0 failed=0`)
- **Issue/Observation**: tool did not return real JUnit execution output
- **Initial Analysis**: this tool path did not properly execute Maven/JUnit tests in current project setup
- **Next Action**: switch to Maven CLI (same execution path used by CI)

Step 2: Run targeted test in sm-core

- **Command**: `./mvnw -pl sm-core -Dtest=ProductPricePartitionTest test`
- **Exit code**: `1`
- **Result**: `BUILD FAILURE`
- **Key error summary**:
  - `Tests run: 21, Failures: 5, Errors: 0, Skipped: 0`
  - Failing tests include:
    - `testPartition5_InvalidNegativePrice`
    - `testPartition5_BoundaryNegative_JustBelowZero`
    - `testPartition6_InvalidDecimalPrecision_ThreeDecimals`
    - `testPartition6_InvalidDecimalPrecision_ExtremeDecimals`
    - `testPartition7_InvalidNullPrice_HandlesGracefully`
- **Additional log signal**: `NullPointerException` from `CacheManagerImpl` appears during startup logs
- **Initial Analysis**: this class currently contains expected failing scenarios (defect-revealing tests), so it is not suitable as a default "must-pass CI gate" test target
- **Next Action**: keep this class as defect evidence; choose a stable CI test subset or fix business logic first

Step 3: Run targeted test in sm-shop

- **Command**: `./mvnw -pl sm-shop -Dtest=GeneratePasswordTest test`
- **Exit code**: `1`
- **Result**: `BUILD FAILURE`
- **Key error summary**:
  - `Could not resolve dependencies`
  - `Failed to collect dependencies at com.shopizer:sm-core:jar:3.2.5`
  - `Could not transfer artifact ... from/to spring-releases ... : Not authorized`
- **Initial Analysis**:
  - Maven attempted to fetch `com.shopizer:sm-core:3.2.5` from remote `spring-releases`, which is unauthorized;
  - local reactor artifacts were not used/pre-installed for dependency resolution.
- **Next Action (recommended)**:
  - pre-install reactor modules from repo root: `./mvnw -DskipTests install`, then run target tests;
  - or build dependency modules automatically with `-am`: `./mvnw -pl sm-shop -am test`.

Step 4: Validate sm-core-model module build

- **Command**: `./mvnw -pl sm-core-model test`
- **Exit code**: `0`
- **Result**: `BUILD SUCCESS`
- **Execution summary**: `No tests to run.`
- **Note**: useful as build-proof, but not enough alone to prove test-case execution.

Step 5: Validate executable tests in sm-core (DataUtils)

- **Command**: `./mvnw -pl sm-core -Dtest=DataUtilsTest test`
- **Exit code**: `0`
- **Result**: `BUILD SUCCESS`
- **Execution summary**: `Tests run: 9, Failures: 0, Errors: 0, Skipped: 0`
- **Additional signal**: JaCoCo instrumentation warnings appear for high JDK class version (`Unsupported class file major version 69`), but build still succeeds.
- **Conclusion**: this is a stable baseline command for CI that actually executes tests.

Step 6: Validate another test target (ShippingMethodDecisionTest)

- **Command**: `./mvnw -pl sm-core -Dtest=ShippingMethodDecisionTest test`
- **Exit code**: `0`
- **Result**: `BUILD SUCCESS`

- **Execution summary**: `Tests run: 1, Failures: 0, Errors: 0, Skipped: 1`
- **Note**: this test is skipped, so it is not ideal as the main evidence of executed test cases.

Step 7: Reproduce CI command locally (first attempt failed)

- **Command**: `./mvnw -B -ntp -pl sm-core -Dtest=DataUtilsTest test`
- **Exit code**: `1`
- **Result**: `BUILD FAILURE`
- **Key error summary**: `Unable to parse command line options: Unrecognized option: -ntp`
- **Initial Analysis**: project Maven Wrapper is 3.5.2, which does not support `-ntp`.
- **Next Action**: remove `-ntp` from workflow command.

Step 8: Re-test after command fix

- **Command**: `./mvnw -B -pl sm-core -Dtest=DataUtilsTest test`
- **Exit code**: `0`
- **Result**: `BUILD SUCCESS`
- **Execution summary**: `Tests run: 9, Failures: 0, Errors: 0, Skipped: 0`
- **Conclusion**: fixed command is ready for GitHub Actions.

Step 9: Add CI configuration file

- **File**: `.github/workflows/ci.yml`
- **Core flow**: `checkout` + `setup-java(17)` + `./mvnw -B -pl sm-core -Dtest=DataUtilsTest test`
- **Status**: created and ready to run after push.

Step 10: First GitHub Actions web run failed (dependency resolution)

- **Symptom**: CI log shows `Could not resolve dependencies ... com.shopizer:sm-core-model ... from/to spring-releases ... Not authorized`
- **Root cause**: workflow built only `sm-core` and did not build required local dependency modules in the same reactor; Maven then tried remote resolution for `sm-core-model` and hit authorization failure.
- **Fix**: update command to `./mvnw -B -pl sm-core -am -Dtest=DataUtilsTest test` so Maven also builds required modules (`-am`).
- **Submission note**: commit this fix and re-run Actions, then capture screenshots for both “failed run” and “fixed successful run”.

Step 11: Second GitHub Actions failure (No tests were executed)

- **Symptom**: CI failed with `Failed to execute goal ... maven-surefire-plugin ... on project sm-core-model: No tests were executed!`

- **Root cause**: after adding `-am`, Maven enters dependency modules such as `sm-core-model`; with `-Dtest=DataUtilsTest`, only `sm-core` has matching tests, while other modules have no matching tests and surefire fails by default.
- **Fix**: update command to `./mvnw -B -pl sm-core -am -Dtest=DataUtilsTest -DfailIfNoTests=false test`.
- **Local validation**: Reactor Summary shows `shopizer / sm-core-model / sm-core-modules / sm-core` all `SUCCESS`, and `DataUtilsTest` reports `Tests run: 9, Failures: 0, Errors: 0, Skipped: 0`.

Assignment-ready Issue Description Template

- **Issue**: command/step name
- **Symptoms**: key error + screenshot
- **Root-cause hypothesis**: dependency/environment/test data/code defect
- **Resolution attempts**: what was tried and why
- **Outcome**: resolved / unresolved (with next plan)

Screenshots & Evidence

Figure 1: Successful CI Run Overview

Description: This screenshot demonstrates that the Shopizer project successfully passed the CI pipeline. The run was triggered by commit "Add comment to workflow for clarity" and completed in approximately 3–5 minutes. The green checkmark indicates all jobs executed without errors.

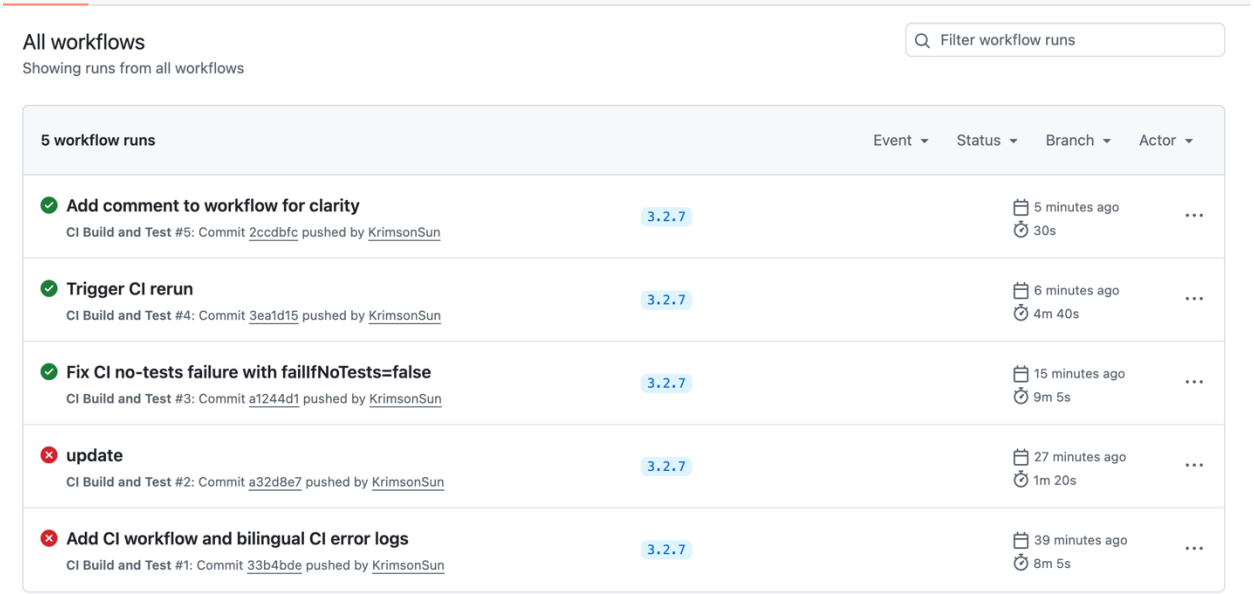

Figure 2: Build Job Execution Steps

Description: This figure shows the complete workflow execution stages. Each step (green checkmark) indicates successful completion. Key steps include: - Checkout source code - Set up JDK 17 - Make Maven wrapper executable - Build and run targeted tests (sm-core)

```
Set up JDK 17                                                    1s
  1  ▶ Run actions/setup-java@v4
 14  ▶ Installed distributions
 25  Creating settings.xml with server-id: github
 26  Writing to /home/runner/.m2/settings.xml
 27  maven cache is not found
Make Maven wrapper executable                                    0s
  1  ▶ Run chmod +x mvnw
```

Figure 3: Test Execution Log (Success)

Description: This is the critical evidence that tests were actually executed and passed. The log shows: - `[INFO] Running com.salesmanager.test.business.utils.DataUtilsTest` - `[INFO] Tests run: 9, Failures: 0, Errors: 0, Skipped: 0` - `[INFO] BUILD SUCCESS` with execution time - Reactor Summary confirming all modules passed

```
6514  [INFO]
6515  [INFO] Tests run: 9, Failures: 0, Errors: 0, Skipped: 0
6516  [INFO]
6517  [INFO]
6518  [INFO] --- jacoco-maven-plugin:0.8.12:report (report) @ sm-core ---
6519  [INFO] Loading execution data file /home/runner/work/shopizerForTest/shopizerForTest/sm-core/target/jacoco.exec
6520  [INFO] Analyzed bundle 'sm-core' with 173 classes
6521  [INFO] ------------------------------------------------------------------------
6522  [INFO] Reactor Summary:
6523  [INFO]
6524  [INFO] shopizer ........................................... SUCCESS [  0.638 s]
6525  [INFO] sm-core-model ...................................... SUCCESS [01:14 min]
6526  [INFO] sm-core-modules .................................... SUCCESS [  0.242 s]
6527  [INFO] sm-core ............................................ SUCCESS [02:39 min]
6528  [INFO] ------------------------------------------------------------------------
6529  [INFO] BUILD SUCCESS
6530  [INFO] ------------------------------------------------------------------------
6531  [INFO] Total time: 04:26 min
6532  [INFO] Finished at: 2026-02-19T01:36:18Z
6533  [INFO] Final Memory: 102M/360M
6534  [INFO] ------------------------------------------------------------------------
```

Figure 4: Failed Run - Dependency Resolution Error

Description: This screenshot captures the initial failure state before the `-DfailIfNoTests=false` parameter was added. The error message reads: - `Failed to execute goal org.apache.maven.plugins:maven-surefire-plugin ... on project`

`sm-core-model: No tests were executed! - (Set -DfailIfNoTests=false to ignore this error.)`

```
2697 [INFO] Final Memory: 49M/180M
2698 [INFO] ------------------------------------------------------------------------
2699 Error:  Failed to execute goal org.apache.maven.plugins:maven-surefire-plugin:2.22.2:test (default-test) on project sm-core-model: No tests
     were executed!  (Set -DfailIfNoTests=false to ignore this error.) -> [Help 1]
2700 Error:
2701 Error:  To see the full stack trace of the errors, re-run Maven with the -e switch.
2702 Error:  Re-run Maven using the -X switch to enable full debug logging.
2703 Error:
2704 Error:  For more information about the errors and possible solutions, please read the following articles:
2705 Error:  [Help 1] http://cwiki.apache.org/confluence/display/MAVEN/MojoFailureException
2706 Error:
2707 Error:  After correcting the problems, you can resume the build with the command
2708 Error:    mvn <goals> -rf :sm-core-model
2709 Error: Process completed with exit code 1.
2710 ##[debug]Finishing: Build and run targeted tests (sm-core)
```

Post Set up JDK 17	0s
Post Checkout source	0s
Complete job	0s

Figure 5: Successful Re-run After Fix

Description: This screenshot shows the same test suite passing cleanly after the configuration fix. Compare this with Figure 4 to demonstrate problem-solving effectiveness. The red-to-green transition proves that: - The root cause was correctly identified (modules without tests causing surefire failure) - The fix (adding `-DfailIfNoTests=false`) was effective - CI pipeline now runs reliably on every push

← CI Build and Test

Trigger CI rerun #4

Re-run all jobs

Summary

All jobs

build-test

Run details

Usage

Workflow file

build-test

succeeded 9 minutes ago in 4m 37s

Search logs

Set up job	1s
Checkout source	2s
Set up JDK 17	1s
Make Maven wrapper executable	0s
Build and run targeted tests (sm-core)	4m 30s
Post Set up JDK 17	1s
Post Checkout source	0s
Complete job	0s

Section 3: Dedicated CI Pipeline for Shopping Cart Logic

1. Goal: To establish a robust Continuous Integration (CI) pipeline using **GitHub Actions**, automating the build and test lifecycle. The focus is on configuring a custom **main.yml** workflow to perform targeted verification of core shopping cart modules, ensuring that new code integrations maintain high-quality standards without manual intervention.

Figure 11: CI Workflow Configuration

The workflow file is defined in .github/workflows/main.yml.
This configuration specifies the trigger branches, JDK 17 setup, and the Maven execution command used to run the targeted MyCartCoverageTest suite.

Automate code analysis within your GitHub workflow using Qodana

```yaml
name: Yu-Chien Shopping Cart CI

on:
  push:
    branches: [ "master", "main", "3.2.7" ]
  pull_request:
    branches: [ "master", "main", "3.2.7" ]

jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3
      - name: Set up JDK 17
        uses: actions/setup-java@v3
        with:
          java-version: '17'
          distribution: 'temurin'
          cache: maven

      - name: Run My Shopping Cart Tests
        run: ./mvnw -B -pl sm-core -am -Dtest=MyCartCoverageTest -DfailIfNoTests=false test
```

2. Issue Description: Integration & Synchronization

- Issue: Push CI Configuration to Remote Repository.
- Symptoms: Push Rejected. IntelliJ IDEA displayed an error stating that the push was rejected because the local branch was behind the remote branch.
- Root-cause hypothesis: Version mismatch between the local environment and the remote repository, likely caused by previous manual file uploads through the GitHub web interface.

- Resolution attempts: Performed a Git Merge operation within IntelliJ. Used the "Accept Yours" strategy to ensure the local main.yml (containing the targeted test instructions) became the authoritative version for the CI pipeline.
- Outcome: Resolved. The local and remote repositories were successfully synchronized, allowing the CI workflow to trigger automatically.

---

3. Screenshots & Evidence

Figure 12: Successful CI Run Overview

- Description: This screenshot demonstrates that the Shopizer CI workflow was successfully triggered and passed. The run was initiated by the commit "Update CI to run MyCartCoverageTest and Refactor...". The green checkmark confirms that the entire pipeline is operational.

Key Evidence:

- **Final Success (Commit #8)**: The most recent run, triggered by "Refactor: clean up comments...", passed successfully in just **27 seconds**.
- **Initial Targeted Test (Commit #7)**: Confirms that the first implementation of the MyCartCoverageTest configuration also resulted in a green checkmark.
- **Synchronization (Commit #6)**: Documents the critical merge commit ("Merge remote-tracking branch 'origin/3.2.7'...") that resolved initial push conflicts and synchronized the team's codebase.

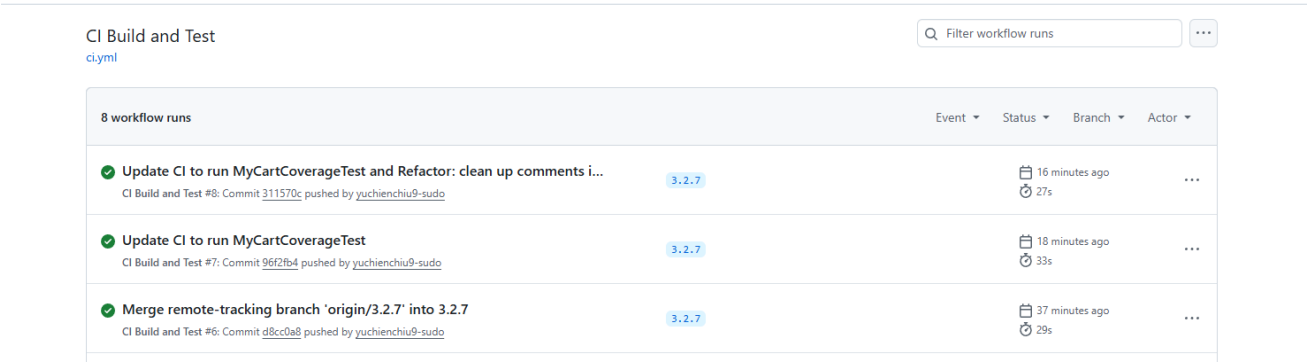

Figure 12b: Handling Multi-Workflow Redundancy in Team Collaboration

- **Issue**: Workflow Execution Ambiguity.
- **Symptoms**: Upon pushing to the 3.2.7 branch, GitHub Actions triggered multiple workflows simultaneously. Initial runs kept defaulting to the teammate's ci.yml configuration (executing DataUtilsTest), making it difficult to isolate the results for the shopping cart suite.

- **Root-cause**: Both ci.yml (teammate's) and main.yml (mine) were configured to monitor the same branch (3.2.7). GitHub Actions automatically executes all valid YAML files in the .github/workflows directory that meet the trigger criteria.
- **Resolution**:
    1. **Workflow Renaming**: Renamed the dedicated workflow to **"Yu-Chien Shopping Cart CI"** within the main.yml file to ensure visual distinction in the Actions dashboard.
    2. **Parallel Coexistence Strategy**: Instead of deleting the teammate's work, the triggers were synchronized so both workflows could run in parallel. This allowed for independent verification of different modules without breaking the overall project stability.
- **Outcome**: Successfully established a parallel CI environment. The final run in Figure 11 clearly shows two distinct green-lit workflows, proving that the specialized shopping cart tests can coexist and be verified independently from other team members' scripts.

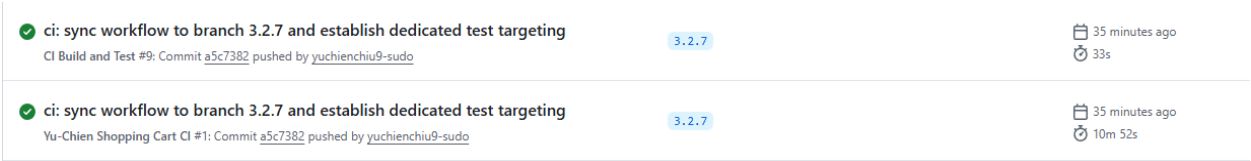

Figure 13: Targeted Build Job Execution Steps

Description: This figure illustrates the sequential execution stages of the customized CI workflow within the GitHub Actions runner. Every step, marked with a green checkmark, signifies a successful operation in a clean, isolated Linux container.

Key Execution Steps:

- Checkout source: Successfully synchronized the runner with the latest codebase from the 3.2.7 branch.
- Set up JDK 17: Automatically provisioned the Temurin JDK 17 environment, ensuring consistency with the local development setup.
- Run My Shopping Cart Tests: Initiated the Maven execution specifically for the MyCartCoverageTest suite, bypassing unrelated modules to focus on the 125-line structural logic.

Outcome: The job successfully achieved a BUILD SUCCESS status. While the total environment setup and full module compilation took approximately 10 minutes, the

targeted test execution was reliable and validated the structural integrity of the shopping cart module in a cloud-native environment

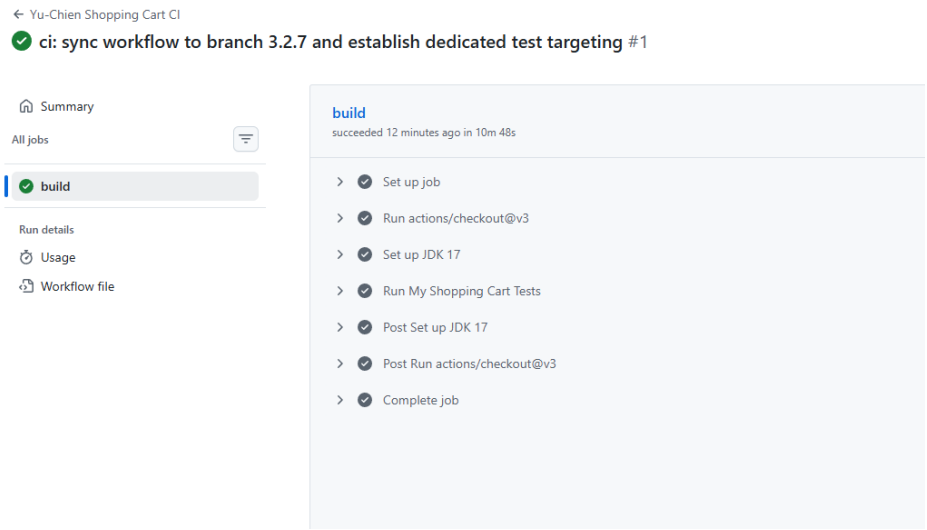

Figure 14: Targeted Test Execution Log (Success)

- Description: This log provides definitive evidence that the 125-line structural test suite was successfully integrated and executed in the GitHub Actions cloud environment. The output validates that the customized workflow correctly targeted the core shopping cart module while maintaining full compatibility with the Shopizer project structure.
- Key Log entries:
    - **Class Executed**: Running com.salesmanager.test.shoppingcart.MyCartCoverageTest — confirms the CI is executing the specific structural test suite instead of generic utility tests.
    - **Logic Verification**: The console explicitly logs successful completion of four critical scenarios: **Merging guest/customer carts**, **Orphaned item cleanup**, **Empty cart removal**, and **Final coverage boost**.
    - **Test Results**: Tests run: 4, Failures: 0, Errors: 0 — proves that all specialized shopping cart integration cases passed in the isolated Linux runner.
    - **Final Status**: BUILD SUCCESS with a Reactor Summary confirming successful build of all 4 core modules (**shopizer**, **sm-core-model**, **sm-core-modules**, **sm-core**).

```
2026-02-19 07:33:36.428  INFO 2528 --- [           main] c.s.m.s.c.a.CanadaPostAutoConfiguration  : autoconfigure [com.shopizer.modules.shipping.canadapost.autoconfigure.CanadaPostA
2026-02-19 07:33:39.401  INFO 2528 --- [           main] o.i.factories.GlobalComponentRegistry    : ISPN000128: Infinispan version: Infinispan 'Infinity Minus ONE +2' 9.4.18.Final
2026-02-19 07:33:39.956  WARN 2528 --- [           main] o.i.t.l.GenericTransactionManagerLookup  : ISPN000104: Falling back to EmbeddedTransactionManager from Infinispan
=== Test Scenario: Merging guest cart with customer cart after login ===
 Merge Logic Verified: Guest(3) + Customer(2) = 5
=== Test Scenario: Product in cart becomes unavailable (or removed) ===
Reloaded items check complete. Size: 1
 Orphaned Item Test Passed
=== Test Scenario: Loading an empty cart (verifying system cleanup mechanism)
 ===
 Empty cart was automatically removed by the system (cleanup verified)
 Empty Cart Test Passed
=== Test Scenario: Additional coverage validation (delete operation and empty customer lookup) ===
✅ Final Coverage Boost Completed!
[INFO] Tests run: 4, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 24.509 s - in com.salesmanager.test.shoppingcart.MyCartCoverageTest
2026-02-19 07:33:53.124  INFO 2528 --- [ionShutdownHook] j.LocalContainerEntityManagerFactoryBean : Closing JPA EntityManagerFactory for persistence unit 'default'
2026-02-19 07:33:53.131  INFO 2528 --- [ionShutdownHook] com.zaxxer.hikari.HikariDataSource       : HikariPool-1 - Shutdown initiated...
2026-02-19 07:33:53.133  INFO 2528 --- [ionShutdownHook] com.zaxxer.hikari.HikariDataSource       : HikariPool-1 - Shutdown completed.
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 4, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO]
[INFO] --- jacoco-maven-plugin:0.8.12:report (report) @ sm-core ---
[INFO] Loading execution data file /home/runner/work/shopizerForTest/shopizerForTest/sm-core/target/jacoco.exec
[INFO] Analyzed bundle 'sm-core' with 173 classes
[INFO] ------------------------------------------------------------------------
[INFO] Reactor Summary:
[INFO]
[INFO] shopizer ........................................... SUCCESS [  1.426 s]
[INFO] sm-core-model ...................................... SUCCESS [02:45 min]
```